

阶段 9：上下文管理与缓存优化实施方案

阶段 9：上下文管理与缓存优化实施方案

状态：Proposed

更新日期：2026-08-06

适用范围：Question Evolution Agent Harness 的 Context Pack、模型调用上下文、Memory 注入和多智能体证据包

目标读者：实现 Context Builder、Planner、Memory 检索、多 Agent 调度和模型调用层的工程师
读完后应能做的事：把当前单一 `context_pack` 拆成可缓存、可裁剪、可审计的上下文结构，并明确缓存 key、失效规则和测试标准。

1. 结论

当前项目已经有短上下文包，能把任务、计划、状态、观察、记忆摘要和决策写入 `context_pack`。这保证了首版 Agent 外壳可恢复、可审计，但还不适合高命中率的模型上下文缓存。

后续优化应先把主控基础上下文拆成 4 层：

代码块

- 1 稳定前缀
- 2 -> 快照前缀
- 3 -> 任务上下文
- 4 -> 动态尾部

模型 prompt 拼装时，在任务上下文之后再插入独立的 `memory_context`，用于承载阶段 4 产出的 Top-K Memory 摘要和 `memory_context_key`。

核心规则：稳定内容放前面，动态内容放后面；缓存 key 只绑定真正影响判断的内容，不绑定运行目录、时间戳和临时状态。

1.1 与相关阶段的职责边界

阶段 9 不重新实现 Memory 系统和多 Agent 调度，也不重新定义 Skills 的业务内容。

职责划分如下：

因此，本阶段的产物是统一的上下文分层、稳定序列化、hash 和 prompt 拼装规则，而不是新的 Memory 发布机制或新的多 Agent 执行器。

2. 当前现状

当前上下文管理主要由 `context.py` 生成短上下文包。它包含：

代码块

```
1  goal
2  task_config
3  hard_constraints
4  selected_search_mode
5  selected_execution_scope
6  plan_revision
7  plan_type
8  budget
9  agent_run_dir
10 experiment_dir
11 current_step_id
12 current_plan_path
13 resume_checkpoint
14 observation_summary
15 memory_summary
16 allowed_tools
17 last_decision
```

当前优点：

- 1. 没有把完整实验日志交给控制层。
- 2. 没有把完整模型响应放进上下文。
- 3. Memory 只进入摘要，不直接控制正式算子和评分。
- 4. 上下文写成文件，方便恢复和审计。

当前问题：

- 1. 稳定字段和动态字段混在一个结构中。

- 2. `agent_run_dir`、`experiment_dir`、`current_step_id`、`last_decision` 等每次运行都变化，不能放进模型 prompt 前缀。
- 3. 没有明确 `context_cache_key`，无法判断两次上下文是否可以复用。
- 4. Memory Top-K、Skill、工具说明、schema 的版本还没有统一进入缓存 key。
- 5. 多 Agent 证据包还没有和主控上下文采用同一套缓存和裁剪规则。

3. 优化目标

本阶段目标不是引入复杂缓存系统，而是先把上下文结构调整可为缓存、可复用、可失效。完成后应达到：

- 1. 模型调用前缀尽量稳定，提高 prompt cache 命中率。
- 2. 不同运行目录、时间戳、当前步骤变化时，不影响稳定前缀缓存。
- 3. 同一策略、同一工具、同一 Skill、同一快照下，可以生成相同 `context_cache_key`。
- 4. Memory 检索结果可复现，相同快照和相同 query 得到相同摘要顺序。
- 5. 多 Agent 子任务只拿到裁剪后的上下文，不继承完整父上下文。
- 6. 所有上下文仍保留证据引用，不能为了缓存而丢失审计链路。

4. 上下文分层

4.1 稳定前缀

稳定前缀适合强缓存。它在大多数运行中保持不变。
内容包括：

禁止放入：运行目录、实验目录、时间戳、当前步骤、最新观察、错误摘要、完整日志。

4.2 快照前缀

快照前缀适合中等粒度缓存。只要快照不变，它就应保持稳定。
内容包括：

代码块

```
1 policy_snapshot_id
2 prompt_snapshot_id
3 operator_snapshot_id
4 memory_snapshot_id
5 tool_registry_version
6 skill_registry_version
7 context_schema_version
```

要求：

1. 快照 ID 必须进入缓存 key。
2. 运行中不得静默切换到最新快照。
3. 恢复运行时必须使用原快照。
4. 找不到原快照时，进入 `blocked` 或显式降级。

4.3 任务上下文

任务上下文适合弱缓存。它和具体用户目标有关，但不会像运行状态一样频繁变化。

内容包括：

代码块

```
1 goal
2 selected_search_mode
3 selected_execution_scope
4 plan_type
5 budget
6 allowed_tools
7 task_config 中影响计划的字段
```

注意：`task_config` 不能整体无筛选进入模型前缀。只保留会影响计划和权限的字段，路径类字段放到动态尾部。

4.4 动态尾部

动态尾部不参与前缀缓存。它放在 prompt 最后，或作为工具参数单独传入。

内容包括：

代码块

```
1 agent_run_id
2 agent_run_dir
```

```
3  experiment_dir
4  current_step_id
5  current_plan_path
6  resume_checkpoint
7  observation_summary
8  last_decision
9  generated_at
10 stdout_summary
11 stderr_summary
12 parse_errors
13 absolute file path
```

规则：动态尾部可以变化，但不能改变稳定前缀和快照前缀的字节内容。

5. 缓存 key 设计

新增 `context_cache_key`，用于判断当前上下文是否可以复用稳定前缀。

建议计算方式：

代码块

```
1  context_cache_key = sha256(
2      context_schema_version
3      + prompt_template_version
4      + skill_registry_version
5      + tool_registry_version
6      + policy_snapshot_id
7      + prompt_snapshot_id
8      + operator_snapshot_id
9      + memory_snapshot_id
10     + selected_search_mode
11     + selected_execution_scope
12 )
```

不得进入 `context_cache_key` 的字段：

代码块

```
1  agent_run_id
2  agent_run_dir
3  experiment_dir
4  current_step_id
5  current_plan_path
6  resume_checkpoint
7  observation_summary
```

```
8 last_decision
9 generated_at
10 stdout_summary
11 stderr_summary
```

原因：这些字段每次运行都可能变，把它们放进 key 会导致缓存几乎无法命中。

6. 序列化规则

所有参与缓存的上下文必须使用稳定序列化。

规则：

1. JSON key 按字典序排序。
2. 列表顺序固定，不能依赖文件系统返回顺序。
3. 工具列表按工具注册表顺序输出。
4. Skill 列表按 `skill_id` 输出。
5. Memory 卡片按检索分数、卡片 ID、版本号稳定排序。
6. 不在稳定前缀中写入当前时间。
7. 不在稳定前缀中写入随机 ID。
8. 不在稳定前缀中写入绝对临时路径。

建议提供统一函数：

代码块

```
1 canonical_json(payload)
```

输出要求：字段稳定、顺序稳定、空白稳定。用于模型 prompt 的版本可以去掉缩进，减少 token 并提高字节稳定性。

7. Memory 上下文缓存

Memory 检索必须独立计算缓存 key。

建议：

代码块

```
1 memory_context_key = sha256(
2     memory_snapshot_id
3     + normalized_query
4     + retrieval_config_version
```

```
5 + top_k
6 )
```

Memory 注入规则：

- 1. 只注入 Top-K 策略摘要，不注入完整历史事实。
- 2. 每条策略摘要必须保留策略卡 ID、版本、证据引用。
- 3. 相同快照、相同 query、相同 top_k 必须得到相同排序。
- 4. 低置信策略不能随机混入 Planner 上下文。
- 5. 完整事实通过 artifact reference 回查，不直接进入 prompt 前缀。

如果 Memory 快照发生变化，`memory_context_key` 和 `context_cache_key` 都必须变化。

8. 多 Agent 上下文缓存

多 Agent 子任务不能继承完整主控上下文。每个专项智能体只拿到裁剪后的上下文。

建议拆分为：

代码块

```
1 advisor_static_prefix
2 advisor_spec_context
3 evidence_pack_slice
4 advisor_dynamic_instruction
```


例子：

代码块

```
1 搜索智能体只拿 016 的 not_applicable 证据引用和日志摘要。
2 路由诊断智能体拿搜索结果和路由记录摘要。
3 建议合成智能体拿多个 advice，不拿完整实验目录。
```

验证类智能体必须新建上下文，不能复用生成类智能体的历史上下文。

9. Prompt 拼装规则

模型调用时按固定顺序拼装：

代码块

```
1  1. stable_prefix
2  2. snapshot_prefix
3  3. task_context
4  4. memory_context
5  5. dynamic_tail
6  6. user_or_system_instruction
```

要求：

1. 1 到 4 尽量稳定，适合缓存。
2. 5 和 6 放最后，允许每次变化。
3. 如果只做报告改写、追加分析或同一证据包复盘，优先复用 1 到 4。
4. 如果快照、工具注册表、Skill 版本变化，必须重新生成 1 到 4。

10. 建议文件结构

后续实现可新增：

代码块

```
1  agent_runtime/context_cache.py
2  agent_runtime/context_layers.py
3  agent_runtime/context_prompt.py
4  schemas/context_pack_v2.schema.json
5  schemas/context_cache.schema.json
6  tests/test_context_layers.py
7  tests/test_context_cache_key.py
8  tests/test_context_prompt_stability.py
9  tests/test_memory_context_cache.py
```

模块职责：

1. `context_layers.py` 负责把上下文拆成稳定前缀、快照前缀、任务上下文和动态尾部。
2. `context_cache.py` 负责稳定序列化、hash 计算和缓存元数据。
3. `context_prompt.py` 负责按固定顺序拼装模型 prompt。
4. `schema` 负责约束上下文字段和缓存元数据。

11. context_pack 升级结构

建议把 context_pack 升级为 v2:

代码块

```
1  {"context_schema_version": "context-pack-v2", "context_cache":  
    {"context_cache_key": "sha256:...", "stable_prefix_hash":  
    "sha256:...", "snapshot_prefix_hash": "sha256:...", "task_context_hash":  
    "sha256:...", "memory_context_hash": "sha256:...", "dynamic_tail_hash":  
    "sha256:..."}, "stable_prefix": {}, "snapshot_prefix": {}, "task_context":  
    {}, "memory_context": {}, "dynamic_tail": {}}
```

兼容要求：第一阶段可以继续输出旧字段，但新增 v2 分层结构；等下游全部改完后，再减少旧字段依赖。

12. 实施步骤

12.1 第一步：上下文字段分类

实现内容：列出当前 context_pack 所有字段，并标记为稳定前缀、快照前缀、任务上下文、Memory 上下文或动态尾部。

验收标准：每个字段都有归属；动态字段不会进入稳定前缀。

12.2 第二步：实现稳定序列化和缓存 key

实现内容：新增稳定 JSON 序列化、hash 计算、context_cache_key 生成。

验收标准：同一输入重复生成完全相同 hash；仅运行目录变化时，稳定前缀 hash 不变。

12.3 第三步：输出 context_pack_v2

实现内容：在保留旧 context_pack 字段的同时，新增分层结构和缓存元数据。

验收标准：旧测试不破坏；新测试能读取分层结构。

12.4 第四步：接入 Memory 上下文缓存

实现内容：接收阶段 4 输出的 Memory Top-K 摘要，为其生成或校验 memory_context_key，并保证排序稳定。

验收标准：相同 Memory 快照、query、retrieval_config_version 和 top_k 生成相同摘要；快照变化后 key 必须变化；每条摘要保留策略卡 ID、版本和 artifact reference。

12.5 第五步：接入多 Agent 证据包

实现内容：接收阶段 7 输出的证据包和 AdvisorSpec，专项智能体输入采用同一套裁剪、hash 和动态尾部规则。

验收标准：子智能体不能拿到完整父上下文；同一 AdvisorSpec 和 `evidence_pack_slice` 生成相同 hash；验证类智能体不能复用生成类智能体上下文。

12.6 第六步：模型 prompt 拼装

实现内容：新增固定 prompt 拼装顺序，确保稳定前缀排在动态尾部之前。

验收标准：同一任务多次运行时，prompt 前缀字节一致；动态尾部变化不影响前缀 hash。

13. 必测场景

必须覆盖以下测试：

1. 仅 `agent_run_dir` 不同，`stable_prefix_hash` 不变。
2. 仅 `current_step_id` 不同，`stable_prefix_hash` 不变。
3. 仅 `observation_summary` 不同，`stable_prefix_hash` 和 `snapshot_prefix_hash` 不变。
4. `policy_snapshot_id` 不同，`context_cache_key` 必须变化。
5. `operator_snapshot_id` 不同，`context_cache_key` 必须变化。
6. `skill_registry_version` 不同，`context_cache_key` 必须变化。
7. 相同 Memory 快照和 query，Memory 摘要顺序一致。
8. Memory 快照变化后，`memory_context_key` 必须变化。
9. 工具列表输入顺序不同，但注册表顺序相同，稳定前缀输出一致。
10. 动态尾部包含路径、时间戳或错误摘要时，不得进入稳定前缀。
11. 子智能体证据包不包含完整实验目录和完整模型响应。
12. 验证类子智能体不能复用生成类子智能体上下文。

14. 最终验收标准

本阶段完成后，应满足：

1. 上下文被明确拆为稳定前缀、快照前缀、任务上下文、Memory 上下文和动态尾部。
2. 所有缓存相关结构都有 hash 和版本字段。
3. 动态字段不会污染模型 prompt 前缀。
4. 快照变化会触发缓存失效。
5. Memory 检索可复现，不能随机改变上下文摘要。
6. 多 Agent 子任务只能读取裁剪后的上下文。

7. 报告、复盘、Planner、Replanner 和多 Agent advice 都能使用同一套上下文分层规则。
8. 上下文优化不降低审计能力，所有摘要仍能追溯到原始 artifact reference。

15. 暂不实施内容

以下内容不进入本阶段：

1. 引入外部分布式缓存服务。
2. 把完整实验历史或完整 Memory 放进模型前缀。
3. 为了命中缓存而删除证据引用。
4. 让子智能体共享完整父智能体上下文。
5. 第一版实现 Fork Subagent。
6. 根据缓存命中结果自动改变正式实验策略。

这些内容容易破坏可审计性或引入额外复杂度。当前阶段先完成上下文分层、稳定序列化和缓存 key 管理。